BEYOND THE DESKTOP: MOBILE & EMBEDDED OPERATING SYSTEMS

Architecture, Challenges, and Convergence

PRESENTED BY

YUE CHENGHAO (227154)
HUA JIE
WANG KAILUN

THE CORE CONFLICT: WHY NON-CONVENTIONAL OS?

Desktop Paradigm

POWER SOURCE

Unlimited Wall Power (High TDP)

INPUT METHOD

Precision Mouse & Keyboard

CONNECTIVITY

Stable Ethernet / High-speed Wi-Fi

Mobile Reality

POWER SOURCE

Strict Battery Constraints (Low TDP)

INPUT METHOD

Imprecise Touch (Fat Finger)

CONNECTIVITY

Unstable Cellular / Variable Latency

> **"Constraints Define Architecture: Mobile OS must be designed from the ground up for efficiency and responsiveness."**

THE TALE OF TWO GIANTS: ANDROID VS IOS

ANDROID

~72%

GLOBAL MARKET SHARE (2025)

PHILOSOPHY

Open Ecosystem, Diverse Hardware

IOS

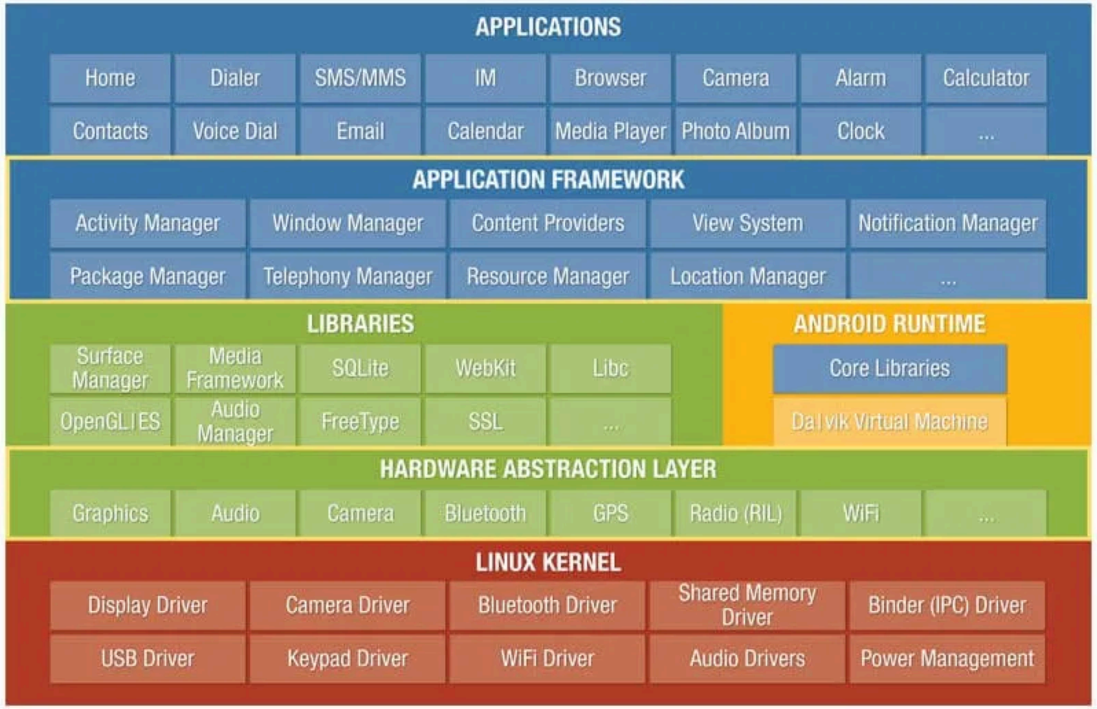
~28%

GLOBAL MARKET SHARE (2025)

PHILOSOPHY

Walled Garden, Integrated Experience

ANDROID ARCHITECTURE: THE "UNIVERSAL TRANSLATOR"



LAYERED STACK: APPS TO LINUX KERNEL

CORE COMPONENT

HAL (Hardware Abstraction Layer)

Acts as a **Universal Translator**. It allows the Android framework to communicate with diverse hardware without exposing manufacturer-specific driver source code (GPL compliance).

COMMUNICATION

Binder IPC Mechanism

A high-performance Inter-Process Communication system. Unlike standard Linux pipes, Binder is **optimized for mobile**, minimizing memory overhead and latency.

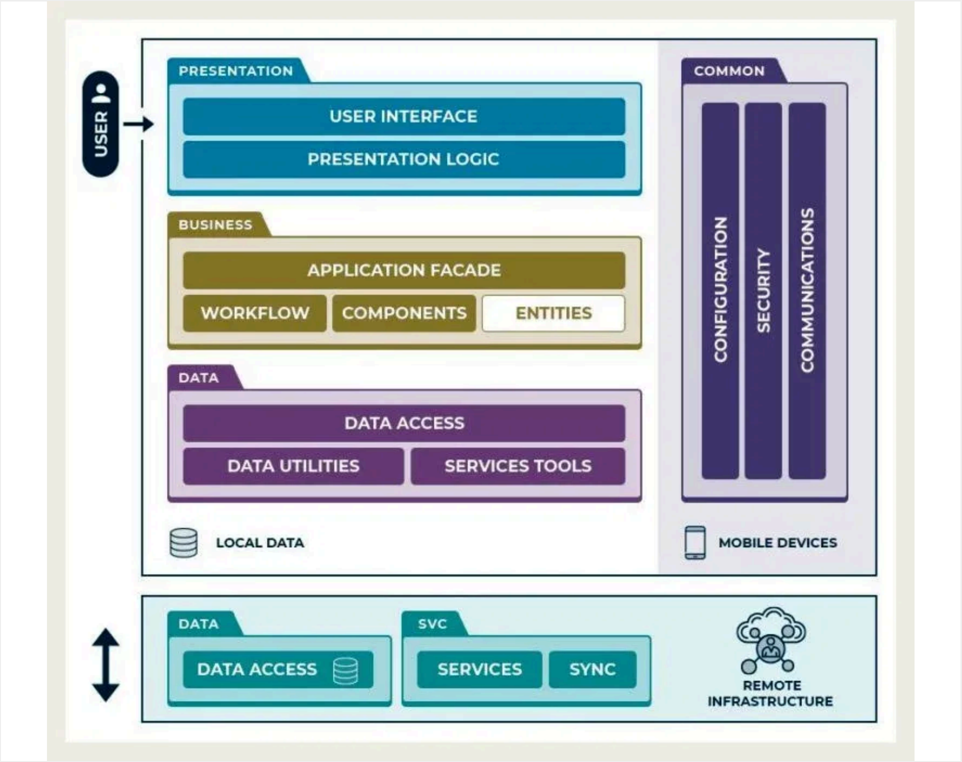
IOS ARCHITECTURE: THE "SPEED DEMON"

UI PRIORITY RENDERING

In iOS, the User Interface (UI) thread is given the **highest system priority**. When a touch event is detected, the system suspends background tasks to ensure a smooth response, creating the perception of superior speed.

STRICT SANDBOXING

Every application operates within its own **isolated container**. Apps cannot access data from other apps directly, ensuring strong security and preventing cross-app interference.



CONCEPTUAL MOBILE ARCHITECTURE: UI & SERVICES FOCUS

THE PRICE OF FREEDOM: FRAGMENTATION

THE CHALLENGE

VERSION SCATTERING

Android's openness leads to extreme fragmentation. Users are scattered across multiple versions (Android 12-15), creating a "nightmare" for developers who must support thousands of device configurations.

~27%

ON ANDROID 14 (2025)

90%+

IOS LATEST VERSION ADOPTION

THE SOLUTION

PROJECT TREBLE

Google's architectural overhaul to simplify updates. It introduces a **Vendor Interface** that separates the core Android OS framework from the device-specific low-level software.

KEY BENEFIT: DECOUPLING

Manufacturers can update the Android OS without waiting for silicon vendors (like Qualcomm) to update their specific drivers, significantly speeding up the update cycle.

DEFINING THE INVISIBLE: WHAT IS EMBEDDED OS?

GENERAL PURPOSE OS

The Swiss Army Knife

Designed to do everything for everyone. Versatile, feature-rich, but carries significant overhead for tasks it wasn't specifically built for.

EMBEDDED OS

The Scalpel

Designed for one specific task. Minimalist, highly specialized, and optimized to perform with absolute precision and efficiency.

DEDICATED

Built for specific hardware and tasks.

RELIABLE

Runs 24/7 for years without crashing.

EFFICIENT

Extreme performance on minimal resources.

THE "TIME" CONCEPT: RTOS & DETERMINISM

CORE DEFINITION

What is "Real-Time"?

Real-time doesn't just mean "fast". It means **Deterministic**. The system must guarantee that a task will be completed within a strictly defined time constraint, regardless of system load.

LIFE-CRITICAL APPLICATIONS

When Timing is Everything

- Airbag Deployment (Milliseconds)
- Heart Pacemakers & Medical Robots

In these scenarios, a late response is as bad as no response at all.

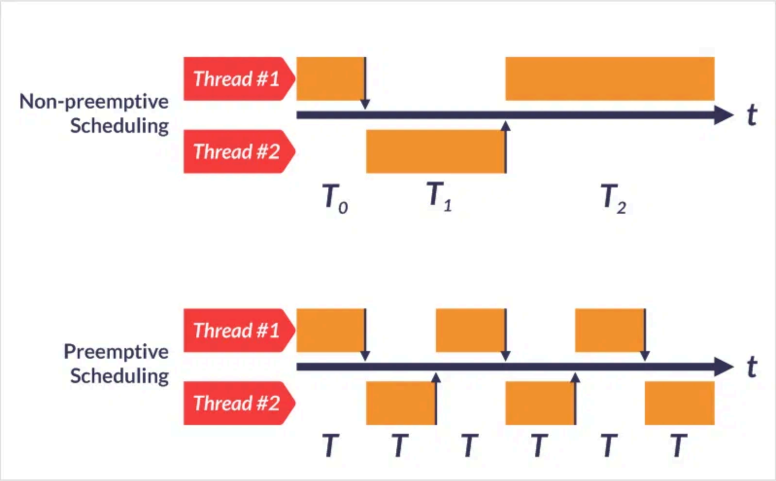
GPOS (BEST EFFORT) VS RTOS (DETERMINISTIC)

UNDER THE HOOD: RTOS SCHEDULING & CONSTRAINTS

SCHEDULING STRATEGY

PREEMPTIVE PRIORITY

Unlike General OS (GPOS) which focuses on fairness, RTOS uses **Preemptive Scheduling**. High-priority tasks (e.g., brakes) can immediately interrupt and "kick out" low-priority tasks (e.g., radio) with microsecond latency.



DETERMINISTIC TASK EXECUTION FLOW

RESOURCE MANAGEMENT

MEMORY: STATIC ALLOCATION

To prevent memory fragmentation and unpredictable crashes, RTOS often avoids dynamic allocation (`malloc`). Memory is **statically allocated** at compile time for maximum reliability.

POWER: TICKLESS IDLE

Instead of a constant "tick" interrupt, **Tickless Idle** allows the CPU to enter deep sleep until the next scheduled event, enabling devices to run on coin batteries for years.

THE CONVERGENCE: "A COMPUTER ON WHEELS"

THE USER EXPERIENCE

MOBILE ECOSYSTEM

Modern drivers expect the richness of a smartphone: Google Maps, Spotify, and voice assistants. This requires a high-level, app-friendly OS like **Android Automotive**.

THE MACHINE CONTROL

EMBEDDED SAFETY

The vehicle's core functions—brakes, steering, and engine control—demand absolute reliability and real-time determinism. This requires a robust **RTOS**.

"The Ultimate Conflict: How do we provide a rich multimedia experience without compromising the safety-critical systems that keep the driver alive?"

CRITICAL RISK: YOU CANNOT RUN BRAKES ON A GENERAL-PURPOSE OS. IF THE MUSIC APP CRASHES, THE CAR MUST STILL STOP.

THE SOLUTION: VIRTUALIZATION & HYPERVISORS

GUEST OS 1: INFOTAINMENT

Android Automotive

Maps, Music, Apps

GUEST OS 2: SAFETY CRITICAL

QNX / RTOS

Brakes, ADAS

VIRTUALIZATION LAYER

Hypervisor (e.g., QNX Hypervisor 8.0)

PHYSICAL LAYER

Vehicle Hardware (SoC)

STRICT ISOLATION

The Hypervisor isolates systems so the RTOS remains unaffected by infotainment issues.

RESOURCE PARTITIONING

Dedicated CPU and memory for safety-critical tasks prevents resource starvation.

CONCLUSION: THE ART OF TRADE-OFFS & FRONTIERS

EDGE INTELLIGENCE

TinyML

Bringing AI to the extreme edge — ML on microcontrollers for smart, offline sensors and appliances.

IOT STANDARDIZATION

Matter Protocol

A unified, secure connectivity standard enabling devices from major vendors to interoperate seamlessly.

MOBILE OS FOCUSES ON THE HUMAN EXPERIENCE.
EMBEDDED OS FOCUSES ON MACHINE CONTROL.

Future engineering is the art of knowing when to demand flexibility and when to ensure absolute reliability.